

B TOPICS

Introduction: The Advantages of Interactive Graphics, Representative Uses of Computer Graphics, Classification of Applications, Development of Hardware and Software for Computer Graphics, Conceptual Framework for Interactive Graphics.

Overview, Scan Converting Lines, Scan Converting Circles, Scan Converting Ellipses.

Graphics Hardware

Hardcopy Technologies, Display Technologies, Raster-Scan Display Systems, The Video Controller, Random-Scan Display Processor, Input Devices for Operator Interaction, Image Scanners, Antialiasing.

Clipping

Cohen- Sutherland Algorithm, (Midpoint Subdivision algorithm.)

Line Drawing Algo

DDALine, Bresenham Line, Midpoint Line Algo

Circle Drawing Algo

Bresenham Circle, Midpoint Circle Algo

Overview of Computer Graphics

(a) Non-Interactive or Passive Computer Graphics:

In non-interactive computer graphics, the picture is produced on the monitor, and the user does not have any controlled over the image, i.e., the user cannot make any change in the rendered image. One example of its Titles shown on T.V.

Non-interactive Graphics involves only one-way communication between the computer and the user, User can see the produced image, and he cannot make any change in the image.

(b) Interactive Computer Graphics:

In interactive Computer Graphics user have some controls over the picture, i.e., the user can make any change in the produced image. One example of it is the ping-pong game.

Interactive Computer Graphics require two-way communication between the computer and the user. A User can see the image and make any change by sending his command with an input device.

Advantages:

1. Higher Quality
2. More precise results or products
3. Greater Productivity
4. Lower analysis and design cost
5. Significantly enhances our ability to understand data and to perceive trends.

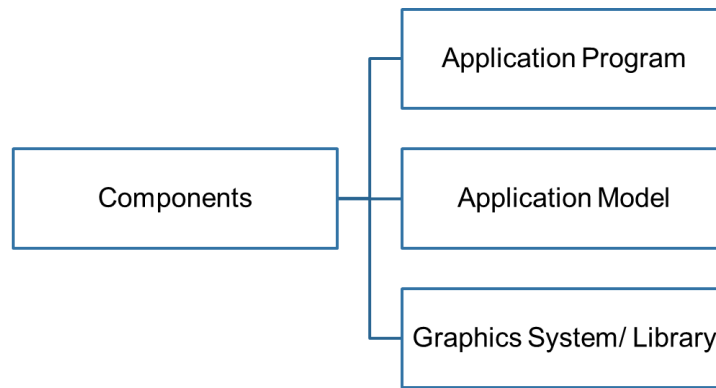
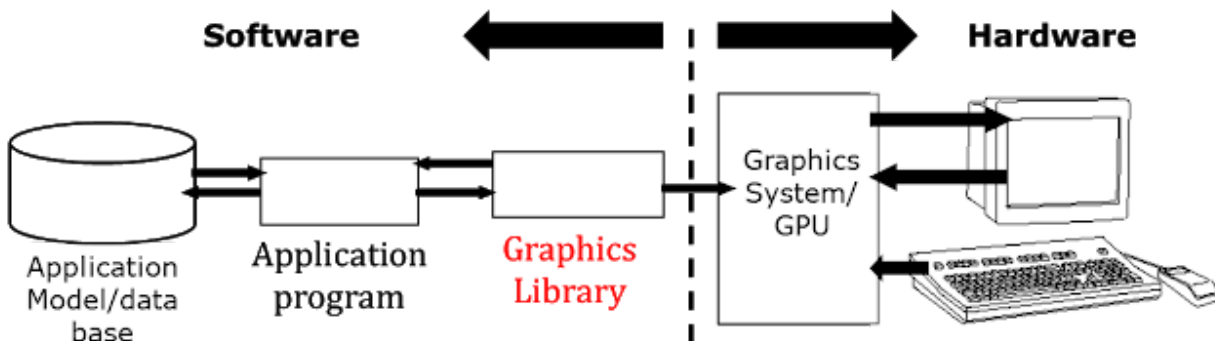


Fig: Conceptual Framework for interactive graphics

- Graphics library/package is intermediary between application and display hardware (Graphics System)
- Application program maps application objects to views (images) of those objects by calling on graphics library. Application model may contain lots of non-graphical data (e.g., non-geometric object properties)
- User interaction results in modification of model and/or image
- This hardware and software framework is more than 4 decades old but is still useful.



Application Model

- represents the data, objects, or scene elements to be drawn

Application Program

- creates, stores into, and retrieves data from the application model
- handles user input
- produces view by sending information to the graphics system
- program calls special functions to perform graphics operations
- functions are part of some graphics software library (written by others)

Graphics System (software & hardware)

- series of graphics output commands
- detailed geometric description of what is to be viewed
- detailed descriptions of how the objects are to appear (also called attributes)
- responsible for actually producing the picture from the descriptions
- passes user's input to the application program for processing
- "intermediary" between the application program and the display hardware

Graphics program developer (application programmer)

- specify what data items or objects are to be generated and represented
- specify how the user interacts with the application program to create/modify the representation
- concerned with creating and editing the model/scene and handling user interaction

- not actually creating views or drawing geometry (handled by the graphics system)

Graphics hardware

Graphics Processing Unit (GPU)

- specialized processor(s) for handling 3D graphics tasks, video decoding, picture output
- texture mapping, rendering, matrix transformations
- programmable shading (processing pixel shading with programs)

Video Memory

- RAM dedicated to the GPU
- designed to be accessed faster than system memory
- may contain texture data, vectors, overlays, GPU programs

Frame buffer

- dedicated memory storing intensity values for each screen pixel
- values retrieved from memory (or buffer) and displayed on screen
- collection of pixel values represent a single frame on the screen
- special type of memory chips enabling fast redisplay of contents
 - size of memory (in bits) for each pixel referred to as the depth
- the greater the depth, the more information associated with each pixel information can include color, transparency, depth,
- additional bits increase number of possible values (colors)

Programming environment

Graphics Software

- user code combined with function calls to perform interactive, 3D graphics tasks
- functions part of software library called an API (Application Programming Interface)
- example includes OpenGL, Microsoft's Direct3D (part of DirectX)
- device-independent packages, promotes application-program portability

Application Programming Interface - API

- Set of functions often collected and residing in a library (a collection of functions or routines)
- Used by programmer to develop applications utilizing some underlying functionality
- Library is a single file on disk containing pre-compiled function definitions
- During compilation, user program must link to library stored on disk
- Header files contain library function declarations (prototypes)
- Header files must be included (#include) in program to declare library functions
- Programmer "sees" only this API - shields the underlying details of both hardware and software of system
- Programs using API's are often called applications or application programs

Graphics API

- Interface between graphics application program and graphics hardware
 - Functions perform some basic three-dimensional computer graphics operations
- Contains different types of functions representing stages of drawing 3D objects

Primitive functions

- Specify what to display

- Define low-level display objects (also called primitives)
- Includes points, lines, line segments, polygons, pixels, text, curves, surfaces

Attribute functions

- Specify how to display primitives
- Govern appearance on the screen
- Can include color, patterns, fonts, texture

Viewing functions

- Specify a synthetic camera viewing the scene
- Can include position, orientation, type of projection, clipping, lens

Transformation functions

- Specify how objects are moved (or transformed)
- Can include scaling, rotation, translation
- Employs heavy use of 4x4 matrix transformations

Input functions

- Specify how to handle user input
- Used for interactive applications
- Specify how to deal with devices such as keyboard, mouse, game controller

Control functions

- Specify general operational functionality
- Examples include initializing the graphics system, communicating with window system, error processing

Application of Computer Graphics

- Computer-Aided Design for engineering and architectural systems etc.
Objects maybe displayed in a wireframe outline form. Multi-window environment is also favored for producing various zooming scales and views. Animations are useful for testing performance.
- Presentation Graphics
To produce illustrations which summarize various kinds of data. Except 2D, 3D graphics are good tools for reporting more complex data.
- Computer Art
Painting packages are available. With cordless, pressure-sensitive stylus, artists can produce electronic paintings which simulate different brush strokes, brush widths, and colors. Photorealistic techniques, morphing and animations are very useful in commercial art. For films, 24 frames per second are required. For video monitor, 30 frames per second are required.
- Entertainment
Motion pictures, Music videos, and TV shows, Computer games
- Education and Training /*
Training with computer-generated models of specialized systems such as the training of ship captains and aircraft pilots.
- Visualization
For analyzing scientific, engineering, medical and business data or behavior. Converting data to visual form can help to understand mass volume of data very efficiently.
- Image Processing
Image processing is to apply techniques to modify or interpret existing pictures. It is widely used in medical applications.
- Medical imaging –
MRIs, CAT scans, etc. - Non-invasive internal examination.

- Graphical User Interface
Multiple window, icons, menus allow a computer setup to be utilized more efficiently.
- **Computer graphics user interfaces** GUIs – A graphic, mouse-oriented paradigm which allows the user to interact with a computer.
- Cartography –
Drawing maps, **Weather Maps** – Real-time mapping, symbolic representations.

Others fields are

- **Satellite Imaging** – Geodesic images.
- **Engineering drawings** – mechanical, electrical, civil, etc. - Replacing the blueprints of the past.
- **Typography** – The use of character images in publishing - replacing the hard type of the past.
- **Architecture** – Construction plans, exterior sketches - replacing the blueprints and hand drawings of the past.
- **Art** – Computers provide a new medium for artists.
- **Training** – Flight simulators, computer aided instruction, etc.
- **Entertainment** – Movies and games.
- **Simulation and modeling** – Replacing physical modeling and enactments

Video Display Devices

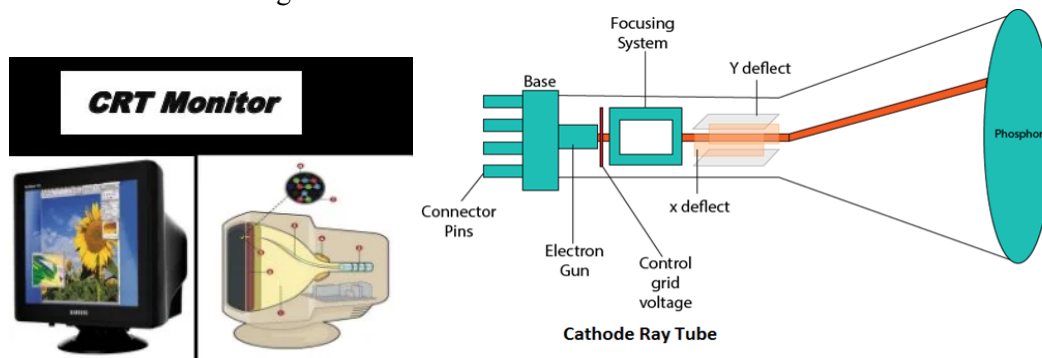
Cathode-Ray Tubes (CRT) - still the most common video display device presently Electrostatic deflection of the electron beam in a CRT

An electron gun emits a beam of electrons, which passes through focusing and deflection systems and hits on the phosphor-coated screen. The number of points displayed on a CRT is referred to as **resolutions** (eg. 1024x768). Different phosphors emit small light spots of different colors, which can combine to form a range of colors. A common methodology for color CRT display is the **Shadow-mask** meth

Illustration of a shadow-mask CRT

The light emitted by phosphor fades very rapidly, so it needs to redraw the picture repeatedly.

There are 2 kinds of redrawing mechanisms: Raster-Scan and Random-Scan



Advantages of CRT

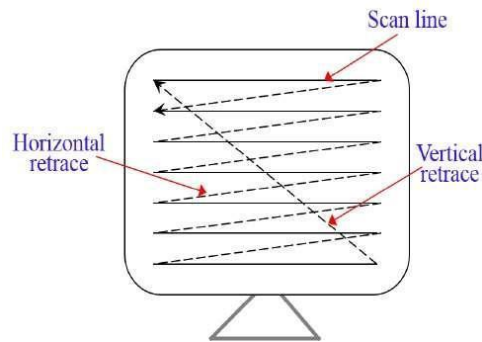
- CRT's are less expensive than other display technologies.
- They operate at any resolution, geometry and aspect ratio without decreasing the image quality.
- CRTs produce the very best color and gray-scale for all professional calibrations.
- Excellent viewing angle.
- It maintains good brightness and gives long life service.

Features of CRT

- The use of CRT technology has quickly declined since the introduction of LCDs but they are still unbeatable in certain ways.
- CRT monitors are widely used in several electrical devices such as computer screens, television sets, radar screens, and oscilloscopes used for scientific and medical purposes.

Raster-Scan

The electron beam is swept across the screen one row at a time from top to bottom. As it moves across each row, the beam intensity is turned on and off to create a pattern of illuminated spots. This scanning process is called refreshing. Each complete scanning of a screen is normally called a **frame**.



The refreshing rate, called the **frame rate**, is normally 60 to 80 frames per second, or described as 60 Hz to 80 Hz.

Picture definition is stored in a memory area called the **frame buffer**. This frame buffer stores the intensity values for all the screen points. Each screen point is called a **pixel** (picture element).

On black and white systems, the frame buffer storing the values of the pixels is called a **bitmap**. Each entry in the bitmap is a 1-bit data which determine the on (1) and off (0) of the intensity of the pixel.

On color systems, the frame buffer storing the values of the pixels is called a **pixmap**. **A pixmap stores and displays a graphical image as a rectangular array of pixel color values. (The term "pixmap" is short for "pixel map".) A pixmap that uses only a single bit to denote the color of each pixel (resulting in a monochrome image) is often referred to as a bitmap. Bitmap is also sometimes used to refer to any pixmap.**

(Though nowadays many graphics libraries name it as bitmap too). Each entry in the pixmap occupies a number of bits to represent the color of the pixel. For a true color display, the number of bits for each entry is 24 (8 bits per red/green/blue channel, each channel 2⁸=256 levels of intensity value, ie. 256 voltage settings for each of the red/green/blue electron guns).

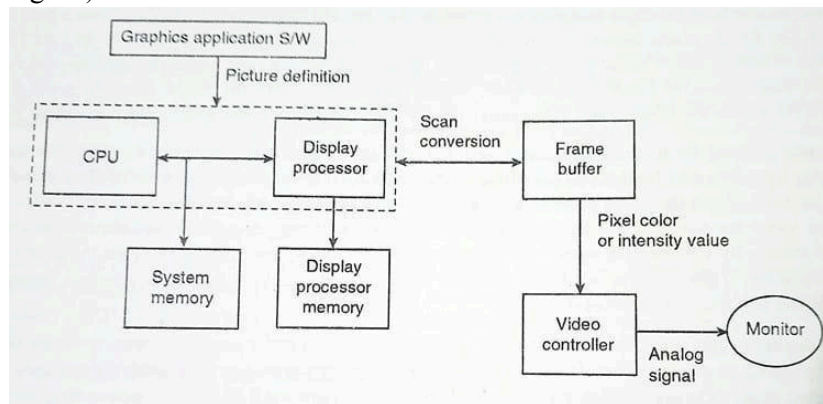
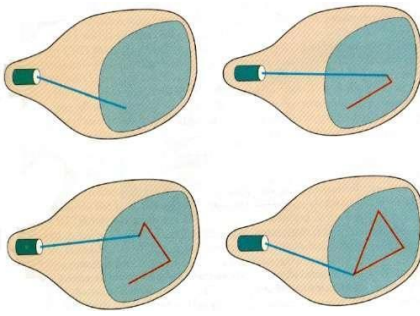


Fig: Architecture of Raster Scan Display

Random-Scan (Vector Display)

The CRT's electron beam is directed only to the parts of the screen where a picture is to be drawn. The picture definition is stored as a set of line-drawing commands in a refresh display file or a refresh buffer in memory.

Random-scan generally have higher resolution than raster systems and can produce smooth line drawings, however it cannot display realistic shaded scenes.



Persistence: Persistence is the duration of phosphorescence. Different kinds of phosphors are available for use in CRT. Besides color, a major difference between phosphor is in their persistence, how they continue to emit light after the electron beam is removed.

Resolution: Used to describe the number of pixels that are used on display image.

Refresh Rate:

To maintain a stable image, the electron beam must sweep the entire surface of the screen and then return to redraw it several times per second. This process is called Refreshing. **Refresh Rate** is the number of times per second that the screen refreshes. It is measured in **Hertz (Hz)**. A high refresh rate is important in providing a clear picture and avoiding eye fatigue. An image appears on screen when electron beams strike the surface of the screen in a zig-zag pattern. A refresh rate is the number of times a screen is redrawn in one second and is measured in Hertz (Hz). Therefore, a monitor with a refresh rate of 85 Hz is redrawn 85 times per second. A monitor should be “flicker-free” meaning that the image is redrawn quickly enough so that the user cannot detect flicker, a source of eye strain. Today, a refresh rate of 75 Hz or above is considered to be flicker-free.

The following formula is used to calculate maximum refresh rates

$$fV = fH / \# \text{ of horizontal lines} \times 0.95$$

fV = vertical scanning frequency (refresh rate) fH = horizontal scanning frequency

Example

A monitor with a horizontal scanning frequency of 96 kHz at a resolution of 1280 x 1024 would have the following refresh rate based on the calculation above.

$$fV = 96,000 / 1024 \times 0.95$$

$$fV = 89.06$$

This figure is rounded down to produce a maximum refresh rate of 89Hz.

If the same monitor is set to a resolution of 1600 x 1200, then the equation will be as follows:

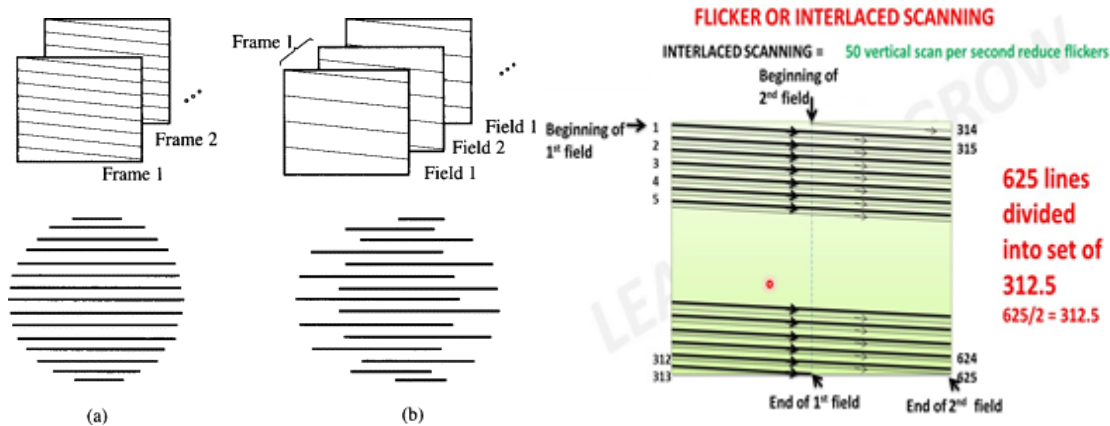
$$fV = 96,000 / 1200 \times 0.95$$

$$fV = 76$$

The maximum refresh rate at this resolution is 76 Hz.

Interlacing:

Interlacing is a technique of improving the picture quality of a video signal without consuming extra bandwidth. In this technique, each frame is displayed in two passes. In the first pass, the beam sweeps across every other scan line from top to bottom. In the second pass, the beam sweeps out the remaining scan lines.



Aspect Ratio:

The **Aspect Ratio** is the ratio of the number of X pixels to the number of Y pixels. The standard aspect ratio for PCs is 4:3. Some common resolutions, the respective number of pixels and standard aspect ratio are given below:

$$\text{Aspect Ratio} = \frac{\text{width unit}}{\text{height unit}}$$

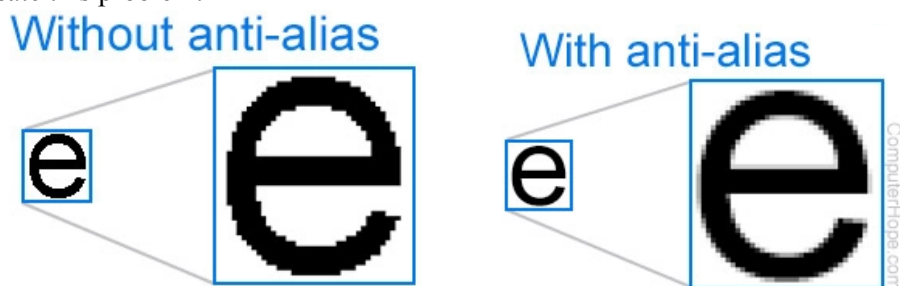
Resolution	Number of Pixels	Aspect Ratio
320x240	76800	4:3
640x480	307200	4:3
800x600	480000	4:3
1024x768	786432	4:3
1280x720	921600	16:9
1920x1080	2073600	16:9

Display Controller

For a raster display device reads the frame buffer and generates the control signals for the screen, ie. the signals for horizontal scanning and vertical scanning. Most display controllers include a **color map** (or video look-up table). The major function of a color map is to provide a mapping between the input pixel value to the output color.

Anti-Aliasing

On dealing with integer pixel positions, jagged or stair step appearances happen very usually. This distortion of information due to under sampling is called aliasing. A number of antialiasing methods have been developed to compensate this problem.



Methods of Antialiasing (AA) –

Aliasing is removed using four methods: Using high-resolution display, Post filtering (Supersampling), Pre-filtering (Area Sampling), Pixel phasing. These are explained as following below.

1. Using high-resolution display:

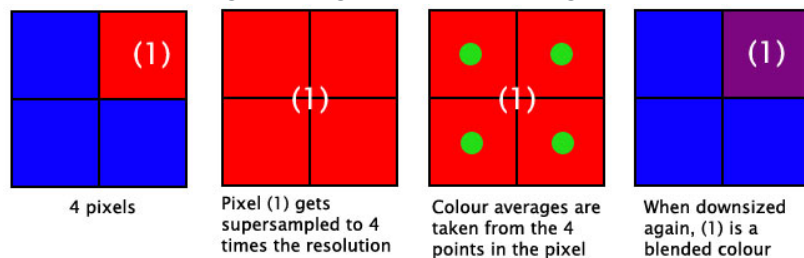
One way to reduce aliasing effect and increase sampling rate is to simply display objects at a higher resolution. Using high resolution, the jaggies become so small that they become indistinguishable by the human eye. Hence, jagged edges get blurred out and edges appear smooth. However there is a limit to how big we can make the frame buffer and still maintaining acceptable refresh rate.

Practical applications:

For example retina displays in Apple devices, OLED displays have high pixel density due to which jaggies formed are so small that they blurred and indistinguishable by our eyes.

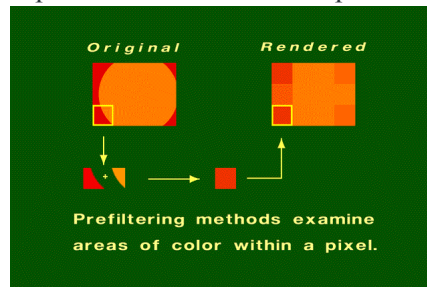
2. Post filtering (Supersampling):

In this method, we are increasing the sampling resolution by treating the screen as if it's made of a much more fine grid, due to which the effective pixel size is reduced. But the screen resolution remains the same. Now, intensity from each subpixel is calculated and average intensity of the pixel is found from the average of intensities of subpixels. Thus we do sampling at higher resolution and display the image at lower resolution or resolution of the screen, hence this technique is called supersampling. This method is also known as post filtration as this procedure is done after generating the rasterized image.



3. Pre-filtering (Area Sampling):

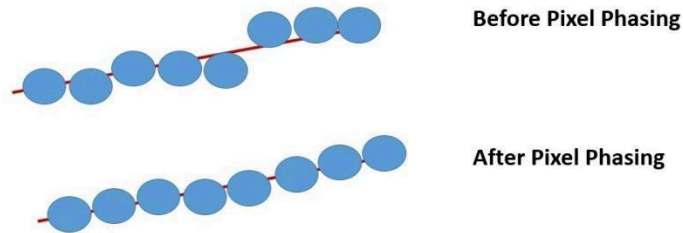
4. In area sampling, pixel intensities are calculated proportional to areas of overlap of each pixel with objects to be displayed. Here pixel color is computed based on the overlap of scene's objects with a pixel area.



For example: Suppose, a line passes through two pixels. The pixel covering bigger portion(90%) of line displays 90% intensity while less area(10%) covering pixel displays 10-15% intensity. If pixel area overlaps with different color areas, then the final pixel color is taken as an average of colors of the overlap area. This method is also known as pre-filtering as this procedure is done BEFORE generating the rasterized image. It's done using some graphics primitive algorithms.

5. Pixel phasing:

It's a technique to remove aliasing. Here pixel positions are shifted to nearly approximate positions near object geometry. Some systems allow the size of individual pixels to be adjusted for distributing intensities which is helpful in pixel phasing.



The primary use of clipping in computer graphics is to remove objects, lines, or line segments that are outside the viewing pane. The viewing transformation is insensitive to the position of points relative to the viewing volume – especially those points behind the viewer – and it is necessary to remove these points before generating the view.

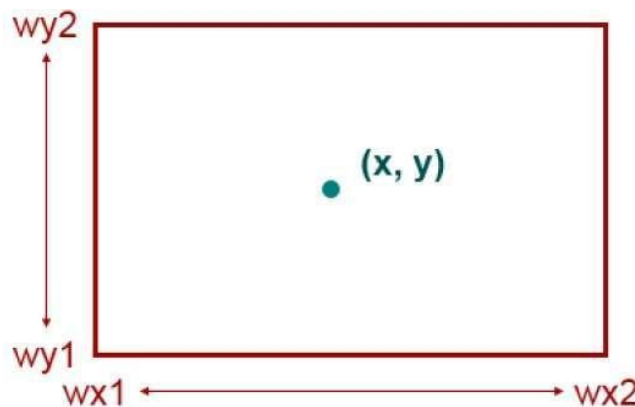
CLIPPING

Point Clipping

Clipping a point from a given window is very easy. Consider the following figure, where the rectangle indicates the window. Point clipping tells us whether the given point X, Y is within the given window or not; and decides whether we will use the minimum and maximum coordinates of the window.

The X-coordinate of the given point is inside the window, if X lies in between $Wx1 \leq X \leq Wx2$. Same way, Y coordinate of the given point is inside the window, if Y lies in between $Wy1 \leq Y \leq Wy2$.

Point Clipping



Line Clipping

The concept of line clipping is same as point clipping. In line clipping, we will cut the portion of line which is outside of window and keep only the portion that is inside the window.

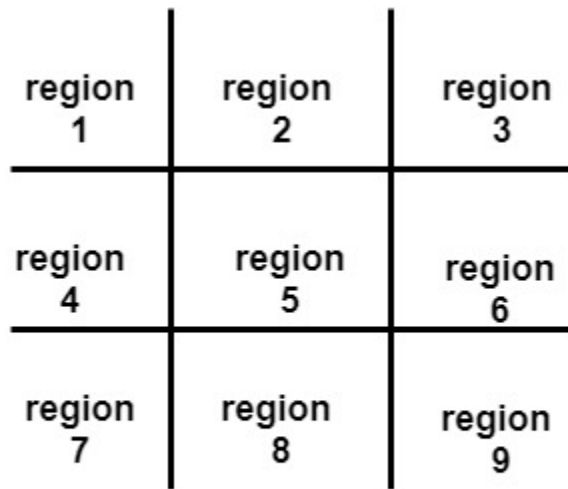
Cohen-Sutherland Line Clippings

This algorithm uses the clipping window as shown in the following figure. The minimum coordinate for the clipping region is $(XWmin, YWmin)$ and the maximum coordinate for the clipping region is $(XWmax, YWmax)$.

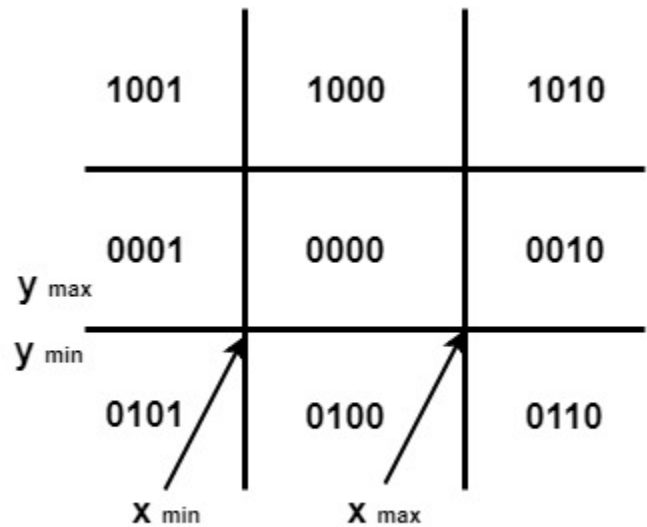
-Sutherland Line Clipping

1001	1000	1010
0001	0000	0010
0101	0100	0110

Line can be partially inside the window We will find intersectionpoint and draw only that portion of line that is inside region.



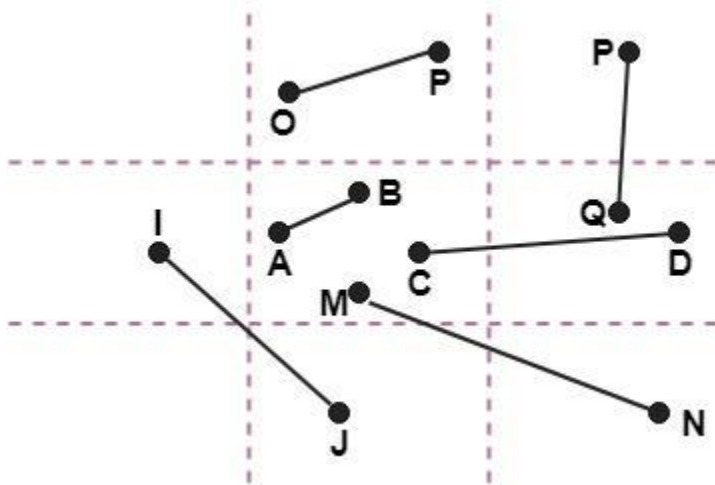
9 region



bits assigned to 9 regions

The center area is having the code, 0000, i.e., region 5 is considered a rectangle window.

Following figure show lines of various types



Line AB is the visible case

Line OP is an invisible case

Line PQ is an invisible line

Line IJ are clipping candidates

Line MN are clipping candidate

Line CD are clipping candidate

HOW TO ASSIGN 0 or 1

For any endpoint (x , y) of a line, the code can be determined that identifies which region the endpoint lies. The code's bits are set according to the following conditions:

- First bit set **1** : Point lies to **left** of window $x < x_{min}$
- Second bit set **1** : Point lies to **right** of window $x > x_{max}$
- Third bit set **1** : Point lies below(**bottom**) window $y < y_{min}$
- fourth bit set **1** : Point lies above(**top**) window $y > y_{max}$

read it as **Bit4 Bit3 Bit2 Bit 1 . TBRL**

We will use 4-bits to divide the entire region. These 4 bits **represent the Top, Bottom, Right, and Left of the region** as shown in the above figure. Here, the TOP and LEFT bit is set to 1 because it is the TOP-LEFT corner.

Algorithm

Step 1 – Assign a region code for each endpoints.

Step 2 – If both endpoints have a region code 0000 then accept this line.

Step 3 – Else, perform the logical AND operation for both region codes.

Step 3.1 – If the result is not 0000, then reject the line.

Step 3.2 – Else you need clipping.

Step 3.2.1 – Choose an endpoint of the line that is outside the window.

Step 3.2.2 – Find the intersection point at the window boundary base on region code.

If a line is clipped case, find an intersection with boundaries of the window

$$m = (y_2 - y_1) / (x_2 - x_1)$$

(a) If bit 1 is "1" line intersects with **left boundary** (__ **0 1**) of rectangle window

$$Y_3 = Y_1 + m(X_{wmin} - X_1)$$

(b) If bit 2 is "1" line intersect with **right boundary** (__ **1 0**)

$$Y_3 = Y_1 + m(X_{wmax} - X_1)$$

(c) If bit 3 is "1" line intersects with **bottom boundary** (**0 1** __)

$$X_3 = X_1 + (Y_{wmin} - Y_1) / m$$

(d) If bit 4 is "1" line intersects with the **top boundary** (**1 0** __)

$$X_3 = X_1 + (Y_{wmax} - Y_1) / m$$

Step 3.2.3 – Replace endpoint with the intersection point and update the region code.

Step 3.2.4 – Repeat step 2 until we find a clipped line either trivially accepted or trivially rejected.

Step 4 – Repeat step 1 for other lines.

Advantage of Cohen Sutherland Line Clipping:

It calculates end-points very quickly and rejects and accepts lines quickly.

It can clip pictures much larger than screen size.

Example of Cohen-Sutherland Line Clipping Algorithm:

Let R be the rectangular window whose lower left-hand corner is at L (-3, 1) and upper right-hand corner is at R (2, 6).

Step 1: Region Codes finding for each point

A (-4, 2) → 0001

E (-2, 3) → 0000

I (-4, 7) → 1001

B (-1, 7) → 1000

F (1, 2) → 0000

J (-2, 10) → 1000

Step 2 : EF satisfies so accept EF line as it is

Category EF i 1 (visible): EF since the region code for both endpoints is 0000.

let there be 3 lines AB with A (-4, 2) and B (-1, 7), E F with E (-2, 3) and F (1, 2) and IJ with I(-4, 7) and J(-2, 10)

Step 3 Now find lines that are outside region and needs clipping

Category2 IJ (not visible): IJ since (1001) AND (1000) = 1000 (which is not 0000).

Category 3 AB (candidate for clipping): AB since (0001) AND (1000) = 0000

Now calculating m for line A B such as $m = (y_2 - y_1) / (x_2 - x_1) = (7 - 2) / (-1 - (-4)) = 5/3$

Now Apply **step 3.2(a)** as per (0001) on A(-4,2) we get Anew with L (-3, 1) and R (2, 6) as min and max coordinate of region.

$$x_3 = x_{\min} = -3 \text{ and } y_3 = Y_1 + m(X_{w\min} - X_1) = 2 + (5/3) (-3 - (-4)) = 2 + (5/3)(1) = 11/3$$

clipped line new A Coordinates will be (-3, 11/3)

Now Apply **step 3.2(d)** as per (1000) on B(-1,7) we get Bnew with L (-3, 1) and R (2, 6) as min and max coordinate of region.

$$y_3 = y_{\max} = 6 \text{ and } x_3 = X_1 + (Y_{w\max} - Y_1) / m = -1 + (6 - 7) / (5/3) = -1 + (3/5)(-1) = (-5 - 3) / 5 = -8/5$$

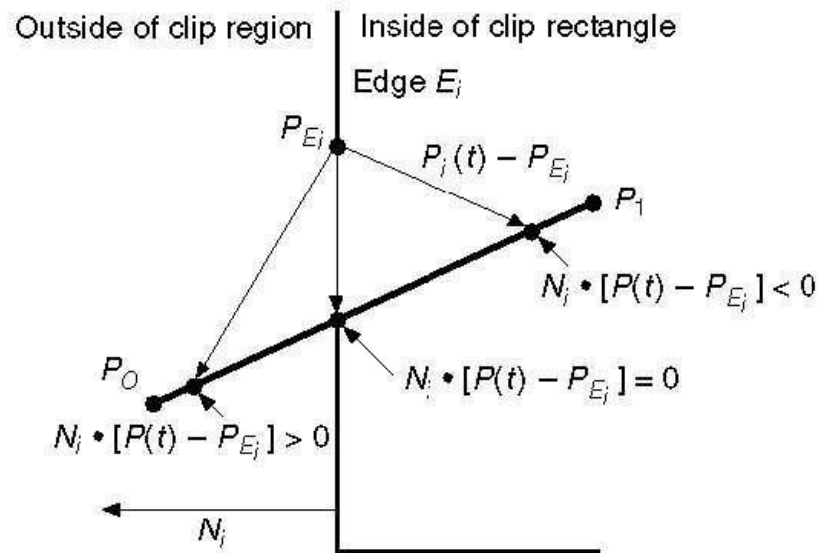
clipped line new B Coordinates will be (-8/5, 6)

Hence New line coordinate will be A'(-3, 3.66) B'(-1.6, 6)

Cyrus-Beck Line Clipping Algorithm

This algorithm is more efficient than Cohen-Sutherland algorithm. It employs parametric line representation and simple dot products.

Cyrus-Beck Line Clipping



Parametric equation of line is –

$$P_0P_1: P(t) = P_0 + t(P_1 - P_0)$$

Let N_i be the outward normal edge E_i . Now pick any arbitrary point P_{Ei} on edge E_i then the dot product $N_i \cdot [P_t - P_{Ei}]$ determines whether the point P_t is “inside the clip edge” or “outside” the clip edge or “on” the clip edge.

The point P_t is inside if $N_i \cdot [P_t - P_{Ei}] < 0$

The point P_t is outside if $N_i \cdot [P_t - P_{Ei}] > 0$

The point P_t is on the edge if $N_i \cdot [P_t - P_{Ei}] = 0$ Intersection point

$$N_i \cdot [P_t - P_{Ei}] = 0$$

$$N_i \cdot [P_0 + t(P_1 - P_0) - P_{Ei}] = 0 \text{ Replacing } P(t) \text{ with } P_0 + t(P_1 - P_0)$$

$$N_i \cdot [P_0 - P_{Ei}] + N_i \cdot t[P_1 - P_0] = 0$$

$$N_i \cdot [P_0 - P_{Ei}] + N_i \cdot tD = 0 \text{ (substituting } D \text{ for } [P_1 - P_0])$$

$$N_i \cdot [P_0 - P_{Ei}] = -N_i \cdot tD$$

The equation for t becomes,

$$t = N_i \cdot [P_0 - P_{Ei}] / (-N_i \cdot D)$$

Here if $N_i \neq 0$ error cannot happen ,

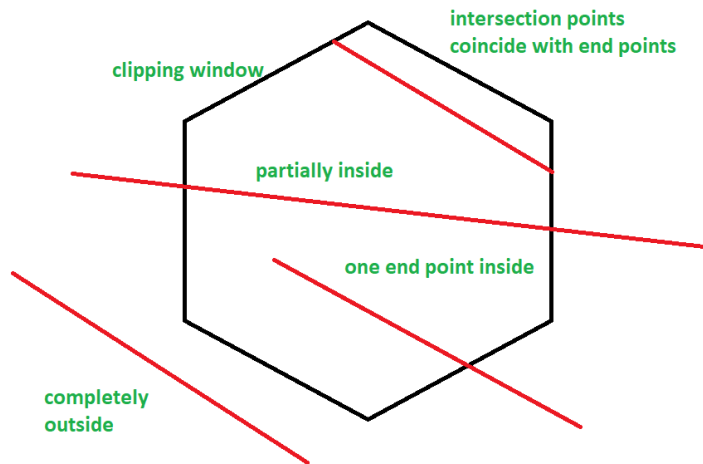
when $D \neq 0$ ($P_1 \neq P_0$)

$N_i \cdot D \neq 0$ (P_0P_1 not parallel to E_i)\

in other terms if t is

- if < 0 , vector pointed towards interior;
- if $= 0$, vector pointed parallel to plane containing p

- if > 0 , vector pointed away from interior.

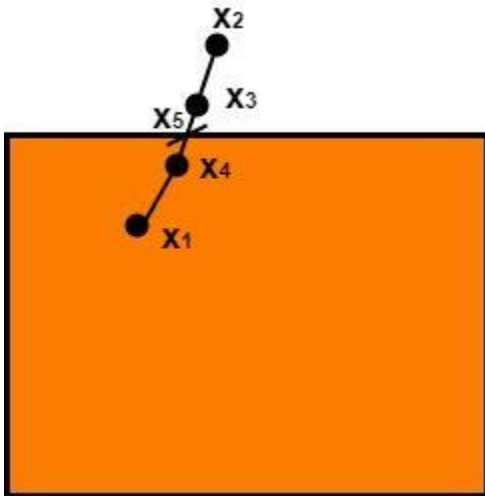


The algorithm is applicable to the “convex polygonal window”. A convex polygonal region having ‘ n ’ vertices $\{P_0, P_1, P_2, \dots, P_{n-1}, P_n, P_0\}$ or lattice points to be specified by the user encloses the convex window region. To be specific about the window we may say that each edge between two points contributes to the boundary of the window under the situation that (when we traverse each edge in anticlockwise manner), the window region lies on left hand side (LHS) of edge. This orientation is preserved and while giving input, the user takes care of the orientation to specify the window region of any arbitrary convex shape.

Mid Point Subdivision Line Clipping Algorithm:

It is used for clipping line. The line is divided in two parts. Mid points of line is obtained by dividing it in two short segments. Again division is done, by finding midpoint. This process is continued until line of visible and invisible category is obtained. Let (x_i, y_i) are midpoint

$$x_m = \frac{x_1 + x_2}{2} \quad y_m = \frac{y_1 + y_2}{2}$$



Step1: Find $\frac{x_2+x_1}{2}$ i. e. $x_3 = \frac{x_2+x_1}{2}$

Step2: Find $x_4 = \frac{x_3+x_1}{2}$

Step3: Find $x_5 = \frac{x_4+x_3}{2}$

x_5 lie on point of intersection of boundary of window.

Advantage of midpoint subdivision Line Clipping:

It is suitable for machines in which multiplication and division operation is not possible. Because it can be performed by introducing clipping divides in hardware.

Algorithm of midpoint subdivision Line Clipping:

Step1: Calculate the position of both endpoints of the line

Step2: Perform OR operation on both of these endpoints

Step3: If the OR operation gives 0000

then

Line is guaranteed to be visible

else

Perform AND operation on both endpoints.

If AND $\neq$ 0000

then the line is invisible

else

AND=6000

then the line is clipped case.

Step4: For the line to be clipped. Find midpoint

$$X_m = (x_1 + x_2) / 2$$

$$Y_m = (y_1 + y_2) / 2$$

X_m is midpoint of X coordinate.

Y_m is midpoint of Y coordinate.

Step5: Check each midpoint, whether it nearest to the boundary of a window or not.

Step6: If the line is totally visible or totally rejected not found then repeat step 1 to 5.

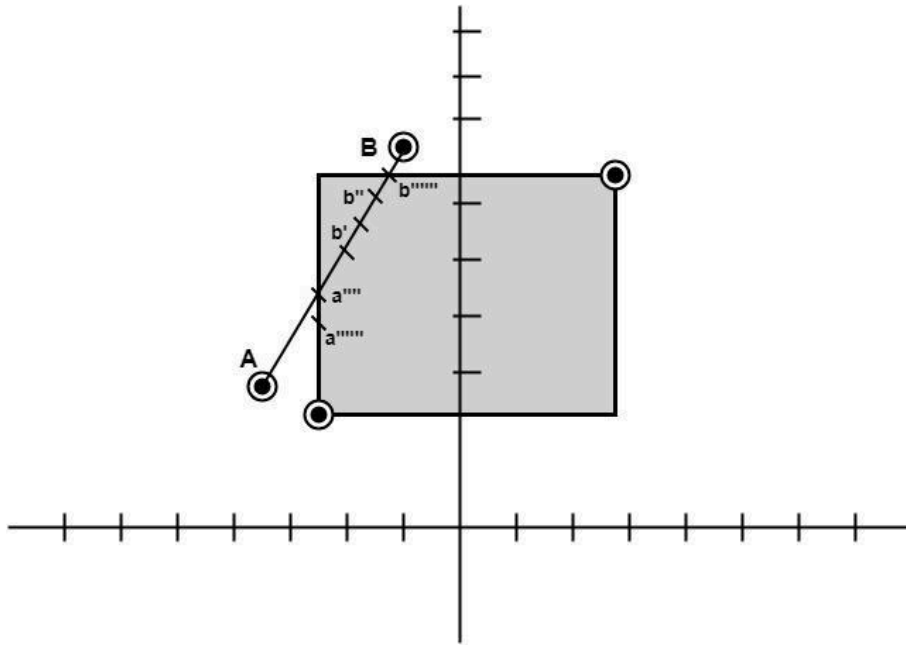
Step7: Stop algorithm.

Example: Window size is (-3, 1) to (2, 6). A line AB is given having co-ordinates of A (-4, 2) and B (-1, 7). Does this line visible. Find the visible portion of the line using midpoint subdivision?

Solution:

Step1: Fix point A (-4, 2) and B (-1, 7).

$$b = \left(\frac{-4 + (-1)}{2}, \frac{2 + 7}{2} \right) = \frac{-5}{2}, \frac{9}{2} = (-2, 4)$$



Step2: Find b'' =mid of b' and b

$$b'' = \left(\frac{-2+(-1)}{2}, \frac{4+7}{2} \right)$$

$$b'' = (-1, 5)$$

So $(-1, 5)$ is better than $(2, 4)$

Find b'' & b''' $(-1, 5)$ & $(-1, 7)$

$$b''' = \left(\frac{-1+(-1)}{2}, \frac{5+7}{2} \right)$$

$$b = (-1, 6)$$

So B''' to B length of line will be clipped from upper side.

Now considered left-hand side portion. A and B''' are now endpoints

Find mid of A and B'''

$$A(-4, 2) \quad B'''(-1, 6)$$

$$a' = \left(\frac{-4+(-1)}{2}, \frac{2+6}{2} \right)$$

$$a' = (-2.5, 4)$$

$$a' = (-2, 4)$$

Now good a (-4, 2) and a' (-2, 4)

$$a'' = \left(\frac{-4+(-2)}{2}, \frac{2+4}{2} \right)$$

$$a'' = (-3, 3)$$

Now find mid of a'' and a

$$a''' = a(-4, 2) \text{ and } a''(-3, 3)$$

$$= \left(\frac{-4+(-3)}{2}, \frac{2+3}{2} \right)$$

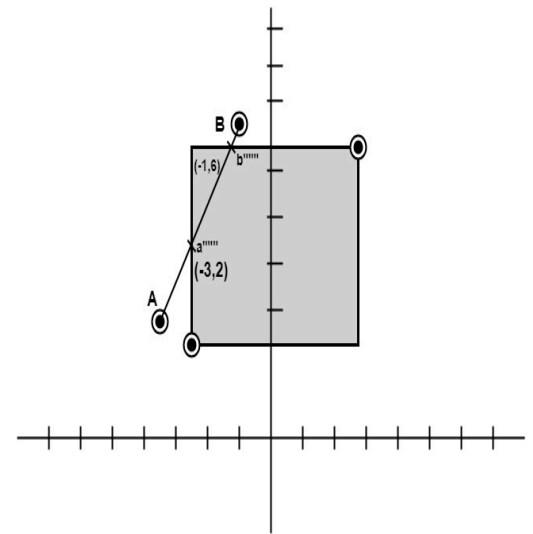
$$= \left(\frac{-7}{2}, \frac{5}{2} \right)$$

$$= (-3.5, 2.5)$$

$$a''' = (-3, 2)$$

So line from A to a''' will be clipped

Line after clipping from both sides will be a''' to b'''



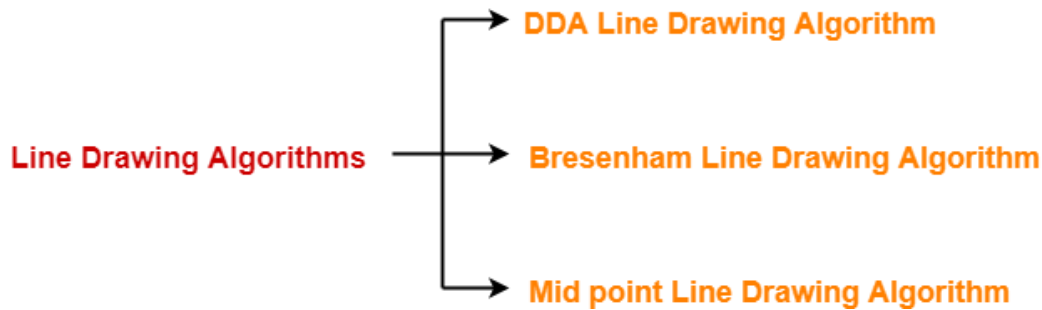
$$a''' = (-3, 2)$$

$$b''' = (-1, 6)$$

LINE GENERATING ALGORITHM

A line connects two points. It is a basic element in graphics. To draw a line, you need two points between which you can draw a line. In the following three algorithms, we refer the one point of line as X_0, Y_0 and the second point of line as X_1, Y_1 .

1. Digital Differential Analyzer (DDA) Line Drawing Algorithm
2. Bresenham Line Drawing Algorithm
3. Mid Point Line Drawing Algorithm



DDA Algorithm

Digital Differential Analyzer (DDA) algorithm is the simple line generation algorithm which is explained step by step here.

Step 1 – Get the input of two end points (X_0, Y_0) and (X_1, Y_1)

Step 2 – Calculate the difference between two end points.

$$dx = X_1 - X_0$$

$$dy = Y_1 - Y_0$$

$$M = dy / dx$$

Step 3 – Based on the calculated difference in step-2, you need to identify the number of steps to put pixel. If $dx > dy$, then you need more steps in x coordinate; otherwise in y coordinate.

```
if (absolute(dx) > absolute(dy))  
    Steps = abs(dx);  
else  
    Steps = abs(dy);
```

Step 4 – Calculate the increment in x coordinate and y coordinate.

$$X_{\text{increment}} = dx / (\text{float}) \text{ steps};$$

$$Y_{\text{increment}} = dy / (\text{float}) \text{ steps};$$

Step 5 – Put the pixel by successfully incrementing x and y coordinates accordingly and complete the drawing of the line.

```
for(int v=0; v < Steps; v++)  
{  
    x = x + Xincrement;  
    y = y + Yincrement;  
    putpixel(Round(x), Round(y));  
}
```

Keep repeating Step-03 until the end point is reached or the number of generated new points (including the starting and ending points) equals to the steps count.

Advantages of DDA Algorithm-

The advantages of DDA Algorithm are-

- It is a simple algorithm.
- It is easy to implement.
- It avoids using the multiplication operation which is costly in terms of time complexity.

Disadvantages of DDA Algorithm-

The disadvantages of DDA Algorithm are-

- There is an extra overhead of using round off() function.
- Using round off() function increases time complexity of the algorithm.
- Resulted lines are not smooth because of round off() function.
- The points generated by this algorithm are not accurate.

EXAMPLE DDA LINE

Problem-01:

Calculate the points between the starting point (5, 6) and ending point (8, 12).

Solution-

Step-01:

Given -

- Starting coordinates = $(X_0, Y_0) = (5, 6)$
- Ending coordinates = $(X_n, Y_n) = (8, 12)$

Step-02:

Calculate ΔX , ΔY and M from the given input.

- $\Delta X = X_n - X_0 = 8 - 5 = 3$
- $\Delta Y = Y_n - Y_0 = 12 - 6 = 6$
- $M = \Delta Y / \Delta X = 6 / 3 = 2$

Step-03:

Calculate the number of steps.

As $|\Delta X| < |\Delta Y| = 3 < 6$, so number of steps = $\Delta Y = 6$

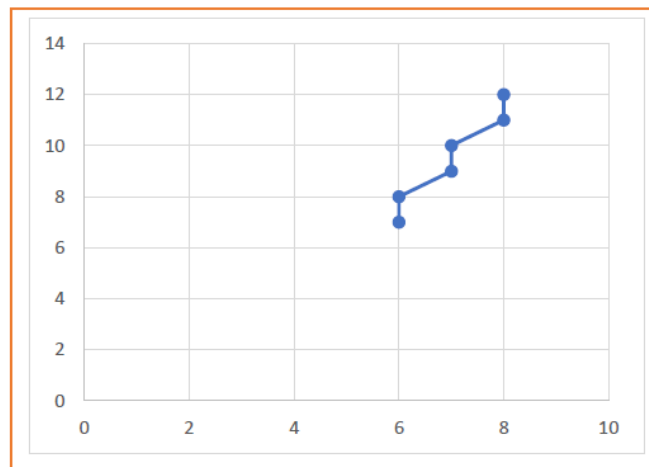
Step-04:

$X_{inc} = \Delta X / (\text{float}) \text{ STEPS} = 3/6 = 0.5$

$X_{inc} = \Delta Y / (\text{float}) \text{ STEPS} = 6/6 = 1$

Now, Step-04 is executed until Step-05 is satisfied.

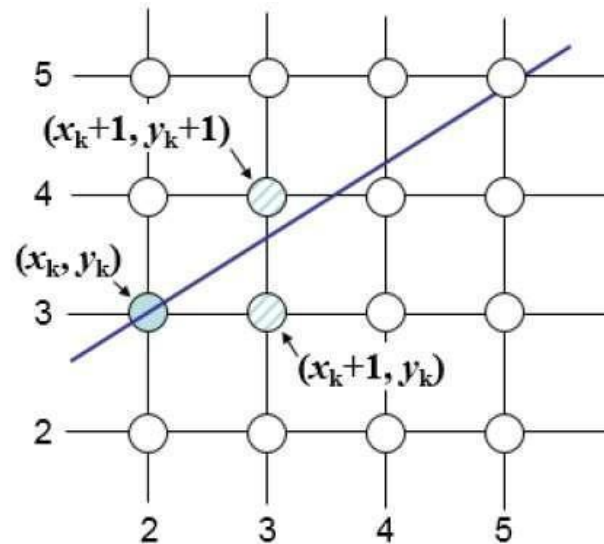
X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)



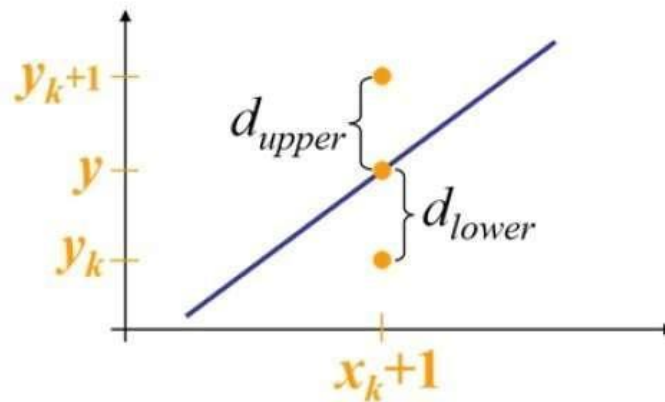
Bresenham's Line Generation

The Bresenham algorithm is another incremental scan conversion algorithm. The big advantage of this algorithm is that, it uses only integer calculations. Moving across the x axis in unit intervals and at each step choose between two different y coordinates.

For example, as shown in the following illustration, from position 2,32,3 you need to choose between 3,33,3 and 3,43,4. You would like the point that is closer to the original line.



At sample position X_{k+1}, Y_{k+1} , the vertical separations from the mathematical line are labelled as d_{upper} and d_{lower} .



From the above illustration, the y coordinate on the mathematical line at x_{k+1} is –
 $Y = mX_{k+1} + b$

So, d_{upper} and d_{lower} are given as follows –

$$d_{lower} = y - y_k$$

$$= m(X_{k+1}) + b - Y_k$$

$$= m(X_{k+1}) + b - Y_k$$

and

$$d_{upper} = (y_{k+1}) - y$$

$$= Y_{k+1} - m(X_{k+1}) - b = Y_{k+1} - m(X_{k+1}) - b$$

You can use these to make a simple decision about which pixel is closer to the mathematical line. This simple decision is based on the difference between the two pixel positions.

$$d_{lower} - d_{upper} = 2m(x_{k+1}) - 2y_k + 2b - 1$$

Let us substitute m with dy/dx where dx and dy are the differences between the end-points.

$$dx(d_{lower} - d_{upper}) = dx(2dy dx(x_{k+1}) - 2y_k + 2b - 1)$$

$$=2dy.x_k-2dx.y_k+2dy+2dx(2b-1)$$

$$=2dy.x_k-2dx.y_k+C$$

So, a decision parameter P_k for the k th step along a line is given by –

$$p_k = dx(d_{lower} - d_{upper})$$

$$=2dy.x_k-2dx.y_k+C=2dy.x_k-2dx.y_k+C$$

The sign of the decision parameter P_k is the same as that of $d_{lower} - d_{upper}$.

If p_k is negative, then choose the lower pixel, otherwise choose the upper pixel.

Remember, the coordinate changes occur along the x axis in unit steps, so you can do everything with integer calculations. At step $k+1$, the decision parameter is given as –

$$p_{k+1} = 2dy.x_{k+1} - 2dx.y_{k+1} + C$$

Subtracting p_k from this we get –

$$p_{k+1} - p_k = 2dy(x_{k+1} - x_k) - 2dx(y_{k+1} - y_k)$$

But, x_{k+1} is the same as $(x_k)+1$. So –

$$p_{k+1} = p_k + 2dy - 2dx(y_{k+1} - y_k)$$

Where, $Y_{k+1} - Y_k$ is either 0 or 1 depending on the sign of P_k .

The first decision parameter p_0 is evaluated at (x_0, y_0) is given as –

$$p_0 = 2dy - dx$$

Now, keeping in mind all the above points and calculations, here is the Bresenham algorithm for slope $m < 1$ –

Step 1 – Input the two end-points of line, storing the left end-point in (x_0, y_0)

Step 2 – Plot the point (x_0, y_0) .

Step 3 – Calculate the constants dx , dy , $2dy$, and $2dy-2dx$ and get the first value for the decision parameter as –

$$p_0 = 2dy - dx$$

Step 4 – At each X_k along the line, starting at $k = 0$, perform the following test –

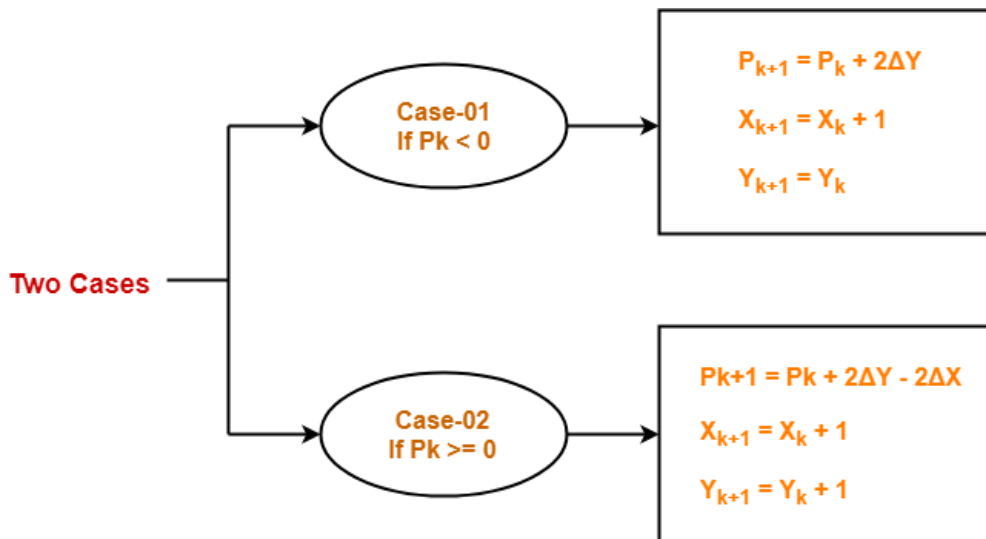
If $p_k < 0$, the next point to plot is (x_{k+1}, y_k) and

$$p_{k+1} = p_k + 2dy$$

Otherwise,

$$(x_k, y_{k+1})$$

$$p_{k+1} = p_k + 2dy - 2dx$$



Step 5 – Repeat step 4 dx–1 times.

For $m > 1$, find out whether you need to increment x while incrementing y each time.

After solving, the equation for decision parameter P_k will be very similar, just the x and y in the equation gets interchanged.

EXAMPLE -BRESENHAM LINE DRAWING ALGORITHM-

Problem-01:

Calculate the points between the **starting coordinates (9, 18)** and **ending coordinates (14, 22)**.

Solution-

Given-

- Starting coordinates = $(X_0, Y_0) = (9, 18)$
- Ending coordinates = $(X_n, Y_n) = (14, 22)$

Step-01:

Calculate ΔX and ΔY from the given input.

- $\Delta X = X_n - X_0 = 14 - 9 = 5$
- $\Delta Y = Y_n - Y_0 = 22 - 18 = 4$

Step-02:

Calculate the decision parameter.

$$\begin{aligned}
 P_k &= 2\Delta Y - \Delta X \\
 &= 2 \times 4 - 5 \\
 &= 3
 \end{aligned}$$

So, decision parameter $P_k = 3$

Step-03:

As $P_k \geq 0$, so case-02 is satisfied.

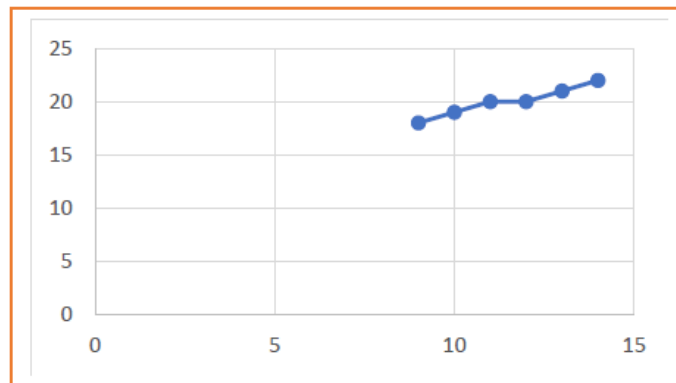
Thus,

- $P_{k+1} = P_k + 2\Delta Y - 2\Delta X = 3 + (2 \times 4) - (2 \times 5) = 1$
- $X_{k+1} = X_k + 1 = 9 + 1 = 10$
- $Y_{k+1} = Y_k + 1 = 18 + 1 = 19$

Similarly, Step-03 is executed until the end point is reached or number of iterations equals to 4 times.

(Number of iterations = $\Delta X - 1 = 5 - 1 = 4$)

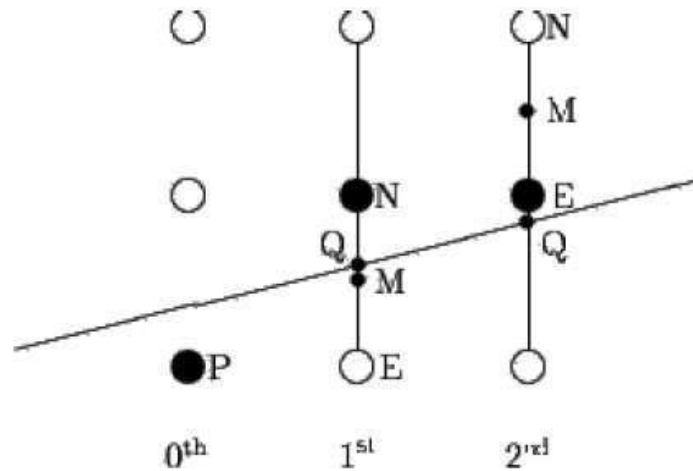
P_k	P_{k+1}	X_{k+1}	Y_{k+1}
		9	18
3	1	10	19
1	-1	11	20
-1	7	12	20
7	5	13	21
5	3	14	22



Mid-Point Algorithm

Mid-point algorithm is due to Bresenham which was modified by Pitteway and Van Aken. Assume that you have already put the point P at x,y coordinate and the slope of the line is $0 \leq k \leq 1$ as shown in the following illustration.

Now you need to decide whether to put the next point at E or N. This can be chosen by identifying the intersection point Q closest to the point N or E. If the intersection point Q is closest to the point N then N is considered as the next point; otherwise E.



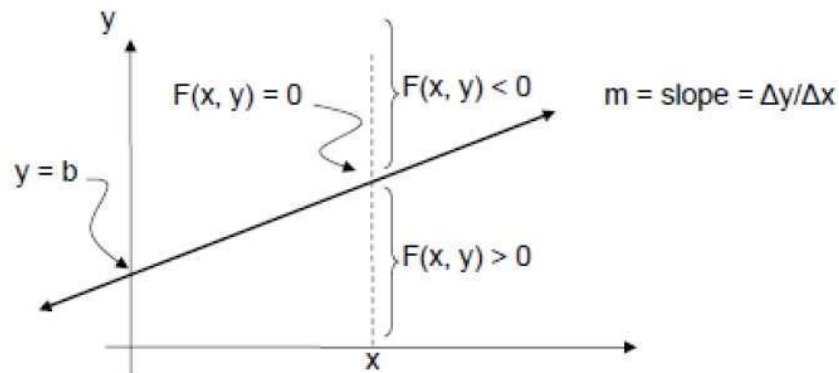
To determine that, first calculate the mid-point $M_{x+1,y+1/2}$. If the intersection point Q of the line with the vertical line connecting E and N is below M, then take E as the next point; otherwise take N as the next point.

In order to check this, we need to consider the implicit equation –

$$F_{x,y,x,y} = mx + b - y$$

For positive m at any given X,

- If y is on the line, then $F_{x,y,x,y} = 0$
- If y is above the line, then $F_{x,y,x,y} < 0$
- If y is below the line, then $F_{x,y,x,y} > 0$



ALGORITHM

Step-01:

Calculate ΔX and ΔY from the given input.

These parameters are calculated as-

- $\Delta X = X_n - X_0$
- $\Delta Y = Y_n - Y_0$

Step-02:

Calculate the value of initial decision parameter and ΔD .

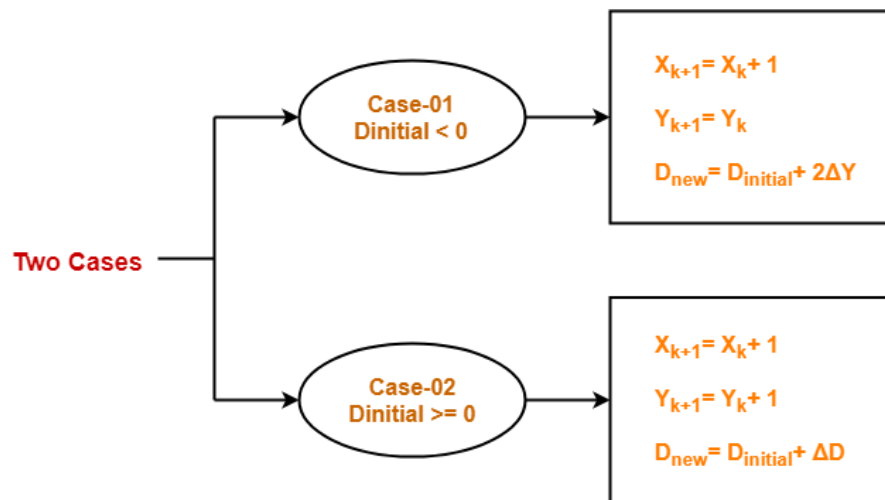
These parameters are calculated as-

- $D_{\text{initial}} = 2\Delta Y - \Delta X$
- $\Delta D = 2(\Delta Y - \Delta X)$

Step-03:

The decision whether to increment X or Y coordinate depends upon the flowing values of D_{initial} .

Follow the below two cases-



Step-04:

Keep repeating Step-03 until the end point is reached.

For each D_{new} value, follow the above cases to find the next coordinates.

EXAMPLE BASED ON MID POINT LINE DRAWING ALGORITHM-

Calculate the points between the starting coordinates (20, 10) and ending coordinates (30, 18).

Solution- Given-

- Starting coordinates = $(X_0, Y_0) = (20, 10)$
- Ending coordinates = $(X_n, Y_n) = (30, 18)$

Step-01:

Calculate ΔX and ΔY from the given input.

- $\Delta X = X_n - X_0 = 30 - 20 = 10$

- $\Delta Y = Y_n - Y_0 = 18 - 10 = 8$

Step-02:

Calculate D_{initial} and ΔD as-

- $D_{\text{initial}} = 2\Delta Y - \Delta X = 2 \times 8 - 10 = 6$
- $\Delta D = 2(\Delta Y - \Delta X) = 2 \times (8 - 10) = -4$

Step-03:

As $D_{\text{initial}} \geq 0$, so case-02 is satisfied.

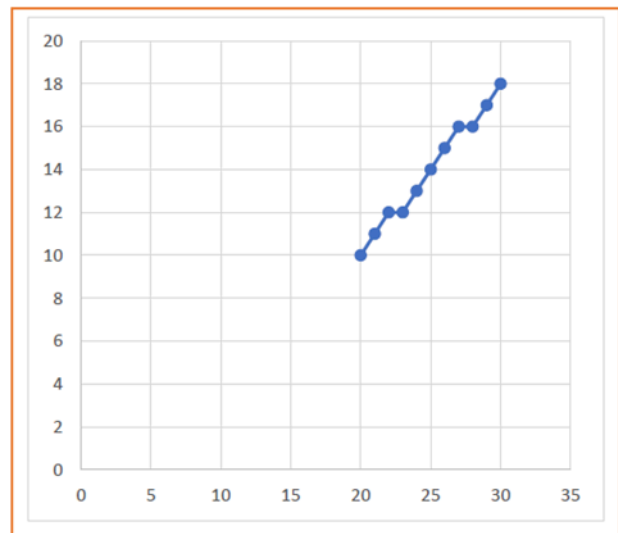
Thus,

- $X_{k+1} = X_k + 1 = 20 + 1 = 21$
- $Y_{k+1} = Y_k + 1 = 10 + 1 = 11$
- $D_{\text{new}} = D_{\text{initial}} + \Delta D = 6 + (-4) = 2$

Step 4

Similarly, Step-03 is executed until the end point is reached.

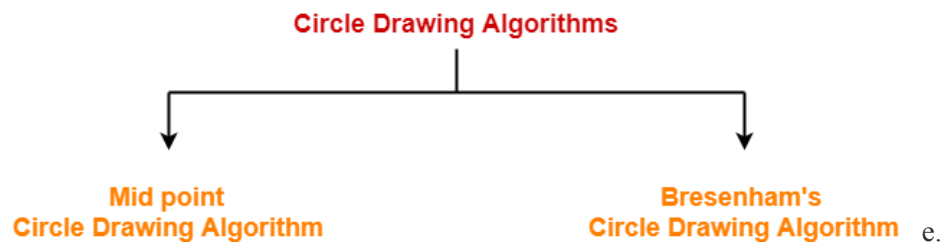
			Y_{k+1}
D_{initial}	D_{new}	X_{k+1}	
		20	10
6	2	21	11
2	-2	22	12
-2	14	23	12



14	10	24	13
10	6	25	14
6	2	26	15
2	-2	27	16
-2	14	28	16
14	10	29	17
10		30	18

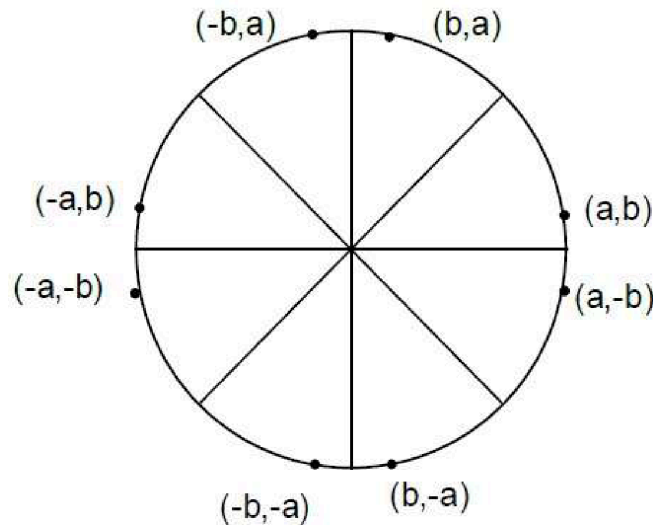
CIRCLE GENERATION ALGORITHM

Drawing a circle on the screen is a little complex than drawing a line. There are two popular algorithms for generating a circle – Bresenham's Algorithm and Midpoint Circle Algorithm. These algorithms are based on the idea of determining the subsequent points required to draw the circle



Let us discuss the algorithms in detail –

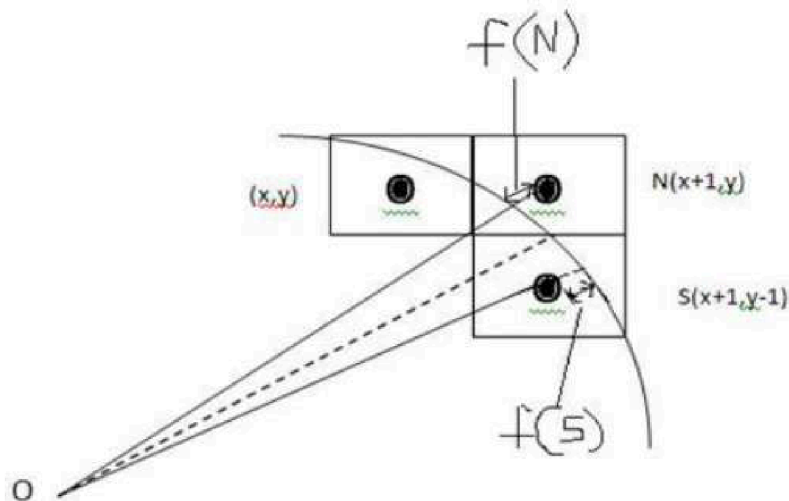
The equation of circle is $X^2 + Y^2 = r^2$, where r is radius.



Bresenham's Algorithm

We cannot display a continuous arc on the raster display. Instead, we have to choose the nearest pixel position to complete the arc.

From the following illustration, you can see that we have put the pixel at X, Y location and now need to decide where to put the next pixel – at $N(X+1, Y)$ or at $S(X+1, Y-1)$



This can be decided by the decision parameter d .

If $d \leq 0$, then $N(X+1, Y)$ is to be chosen as next pixel.

If $d > 0$, then $S(X+1, Y-1)$ is to be chosen as the next pixel.

Algorithm

Step 1 – Get the coordinates of the center of the circle and radius, and store them in x_c , y_c , and R

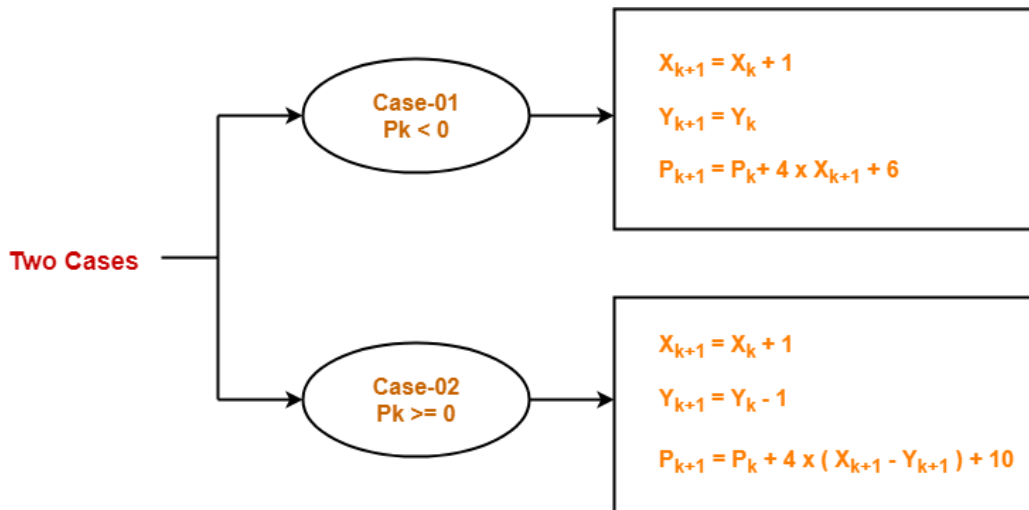
respectively. Set $X=0$ and $Y=R$. (IF X_c AND Y_c NOT ZERO THEM FIRST SUBTRACT THE VALUES AND MAKE THEM ZERO , after that the above algo is valid , later this x_c , y_c to be added to all derived pixel coordinates)

Step 2 – Set decision parameter $P = 3 - 2R$.

Step 3 – Repeat through step-8 while $X < Y$.

Step 4 – Call Draw Circle X, Y, X_c, Y_c .

Step 5 – Increment the value of P .

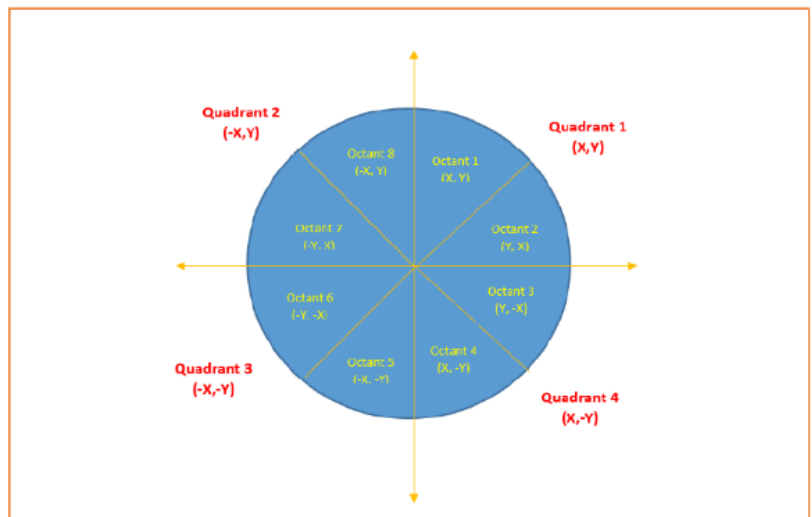


Step 6 – If $P < 0$ then $P_{k+1} = P_k + 4x + 6$.

Step 7 – Else Set $Y = Y + 1$, $P_{k+1} = P_k + 4(X-Y) + 10$.

Step 8 – Call Draw Circle X, Y, X_c, Y_c . Plot eight points by using concepts of eight-way symmetry. The center is at (X_c, Y_c) . Current active pixel is (X, Y) .

```
putpixel (X+Xc, Y+Yc)
putpixel (X+Xc, -Y+Yc)
putpixel (-X+Xc, Y+Yc)
putpixel (-X+Xc, -Y+Yc)
putpixel (Y+Xc, X+Yc)
putpixel (Y+Xc, -X+Yc)
putpixel (-Y+Xc, X+Yc)
putpixel (-Y+Xc, -X+Yc)
```



PRACTICE PROBLEMS BASED ON BRESENHAM CIRCLE DRAWING ALGORITHM-

Problem-01:

Given the centre point coordinates (0, 0) and radius as 8, generate all the points to form a circle.

Solution-

Given-

- Centre Coordinates of Circle $(X_0, Y_0) = (0, 0)$
- Radius of Circle = 8

Step-01:

Assign the starting point coordinates (X_0, Y_0) as-

- $X_0 = 0$
- $Y_0 = R = 8$

Step-02:

Calculate the value of initial decision parameter P_0 as-

$$P_0 = 3 - 2 \times R$$

$$P_0 = 3 - 2 \times 8$$

$$P_0 = -13$$

Step-03:

Call draw circle (0,0,8)

Step-04:

Now find next x, y values

As $P_{\text{initial}} < 0$, so case-01 is satisfied.

Thus,

- $X_{k+1} = X_k + 1 = 0 + 1 = 1$
- $Y_{k+1} = Y_k = 8$
- $P_{k+1} = P_k + 4 \times X_{k+1} + 6 = -13 + (4 \times 1) + 6 = -3$
(if $P_k > 0$ $P_{k+1} = P_k + 4 \times (X_{k+1} - Y_{k+1}) + 10$)

Step-04:

This step is not applicable here as the given centre point coordinates is (0, 0). Call draw circle (0,0,1,8) where following putpixels will be executed:

putpixel (1,8) putpixel (1,-8) putpixel (-1,8) putpixel (-1, -8)
 putpixel (8,1) putpixel (8, -1) putpixel (-8, 1) putpixel (-8,-1)

Step-05:

Step-03 is executed similarly until $X_{k+1} \geq Y_{k+1}$ as follows- (for 1 octant)

P_k	P_{k+1}	(X_{k+1}, Y_{k+1})	ONE OCTANT actual pixel coordinate wrt centre xc,yc ex(2,4)
		(0, 8)	(0+Xc,8+Yc) (2,12)
-13	-3	(1, 8)	(1+Xc,8+Yc) (3,12)
-3	11	(2, 8)	(2+Xc,8+Yc) (4,12)
11	5	(3, 7)	(3+Xc,7+Yc)
5	7	(4, 6)	(4+Xc,6+Yc)
7		(5, 5)	(5+Xc,5+Yc)

Here, all the points have been generated with respect to quadrant-1-

Quadrant-1 (X,Y)	Quadrant-2 (-X,Y)	Quadrant-3 (-X,-Y)	Quadrant-4 (X,-Y)
(0, 8)	(0, 8)	(0, -8)	(0, -8)
(1, 8)	(-1, 8)	(-1, -8)	(1, -8)
(2, 8)	(-2, 8)	(-2, -8)	(2, -8)
(3, 7)	(-3, 7)	(-3, -7)	(3, -7)
(4, 6)	(-4, 6)	(-4, -6)	(4, -6)

(5, 5)	(-5, 5)	(-5, -5)	(5, -5)
(6, 4)	(-6, 4)	(-6, -4)	(6, -4)
(7, 3)	(-7, 3)	(-7, -3)	(7, -3)
(8, 2)	(-8, 2)	(-8, -2)	(8, -2)
(8, 1)	(-8, 1)	(-8, -1)	(8, -1)
(8, 0)	(-8, 0)	(-8, 0)	(8, 0)

Mid Point Algorithm

Step 1 – Input radius r and circle center $(x_{\{c\}}, y_{\{c\}})$ and obtain the first point on the circumference of the circle centered on the origin as

Given-

- Centre point of Circle = (X_0, Y_0)
- Radius of Circle = r

Let $(x_0, y_0) = (0, r)$

Step 2 – Calculate the initial value of decision parameter as

$P\{0\} = 5/4 - r \approx 1.25 - r$

When rounded off

$P = 1 - r$

See the following description for simplification of this equation.

$$f(x, y) = x^2 + y^2 - r^2 = 0$$

$$\begin{aligned} f(x_i - 1/2 + e, y_i + 1) &= (x_i - 1/2 + e)^2 + (y_i + 1)^2 - r^2 \\ &= (x_i - 1/2)^2 + (y_i + 1)^2 - r^2 + 2(x_i - 1/2)e + e^2 \\ &= f(x_i - 1/2, y_i + 1) + 2(x_i - 1/2)e + e^2 = 0 \end{aligned}$$

$$\text{Let } d_i = f(x_i - 1/2, y_i + 1) = -2(x_i - 1/2)e - e^2$$

Thus,

If $e < 0$ then

$d_i > 0$ so choose point $S = (x_i - 1, y_i + 1)$.

$$\begin{aligned} d_{i+1} = f(x_i - 1, y_i + 1) &= ((x_i - 1/2) - 1)^2 + ((y_i + 1) + 1)^2 - r^2 \\ &= d_i - 2(x_i - 1/2) + 1 + 2(y_i + 1) + 1 - r^2 \\ &= d_i + 2(y_i + 1) - x_i + 1 + 1 \end{aligned}$$

If $e \geq 0$ then $d_i \leq 0$ so choose point $T = (x_i, y_i + 1)$

$$d_{i+1} = f(x_i - 1/2, y_i + 1 + 1) = d_i + 2y_{i+1} + 1$$

The initial value of d_i is

$$\begin{aligned} d_0 = f(r - 1/2, 0 + 1) &= (r - 1/2)^2 + 1^2 - r^2 \\ &= 5/4 - r \end{aligned}$$

used if r is an integer}

When point $S = (x_i - 1, y_i + 1)$ is chosen then

$$d_{i+1} = d_i - 2x_{i+1} + 2y_{i+1} + 1$$

When point $T = (x_i, y_i + 1)$ is chosen then

$$d_{i+1} = d_i + 2y_{i+1} + 1$$

Step 3 – At each $X\{K\}$ position starting at $K=0$, perform the following test –
If $P_K < 0$ then next point on circle $(0,0)$ is (X_{K+1}, Y_K) and

$$P_{K+1} = P_K + 2X_{K+1} + 1$$

Else

$$P_{K+1} = P_K + 2X_{K+1} + 1 - 2Y_{K+1}$$

$$\text{Where, } 2X_{K+1} = 2X_K + 2 \text{ and } 2Y_{K+1} = 2Y_K - 2.$$

Step 4 – Determine the symmetry points in other seven octants.

Step 5 – Move each calculate pixel position X, Y onto the circular path centered on $(X_{-}\{C\}, Y_{-}\{C\})$ and plot the coordinate values.

$$X = X + X_C, Y = Y + Y_C$$

Step 6 – Repeat step-3 through 5 until $X \geq Y$.

Step-02:

Calculate the value of initial decision parameter P_0 as-

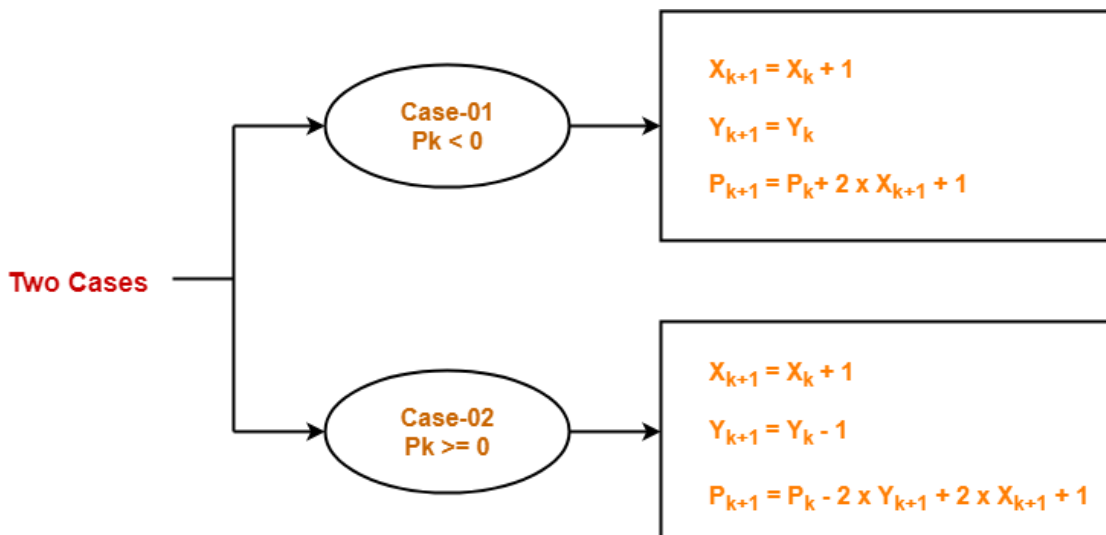
$$P_0 = 1.25 - R \text{ or } 1 - R \text{ (if rounded off)}$$

Step-03:

Suppose the current point is (X_k, Y_k) and the next point is (X_{k+1}, Y_{k+1}) .

Find the next point of the first octant depending on the value of decision parameter P_k .

Follow the below two cases-



Step-04:

If the given centre point (X_0, Y_0) is not $(0, 0)$, then do the following and plot the point-

- $X_{\text{plot}} = X_c + X_0$
- $Y_{\text{plot}} = Y_c + Y_0$

Here, (X_c, Y_c) denotes the current value of X and Y coordinates.

Step-05:

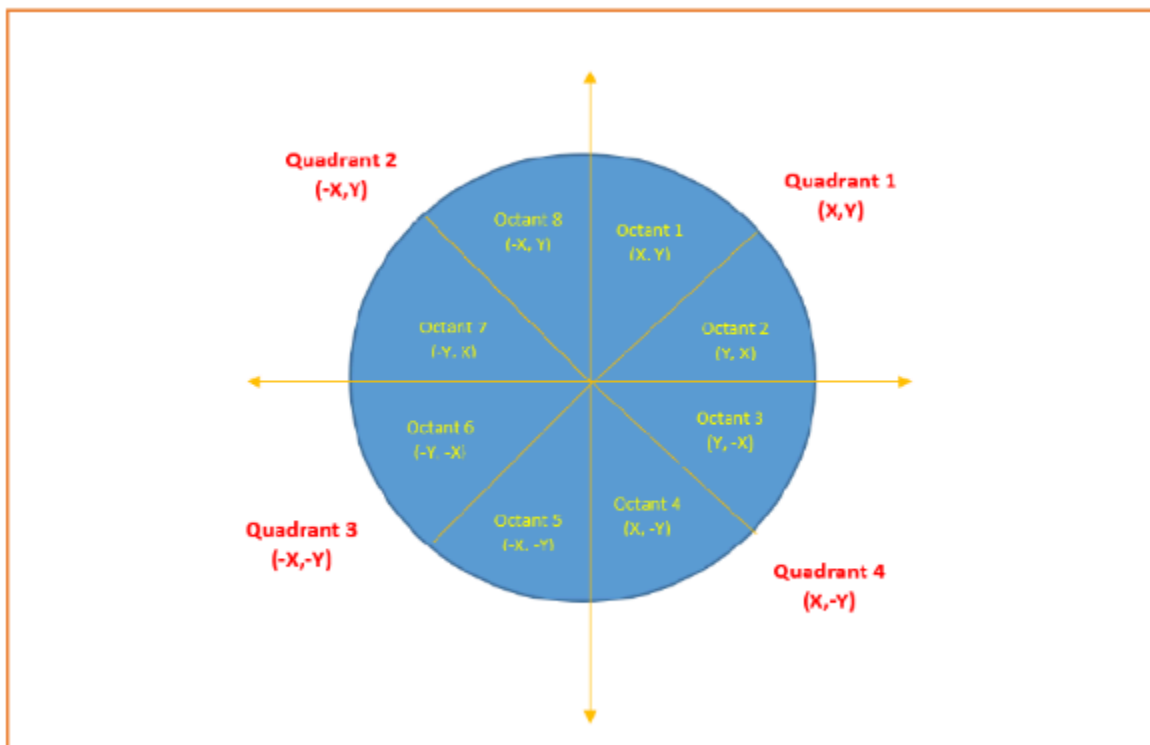
Keep repeating Step-03 and Step-04 until $X_{\text{plot}} \geq Y_{\text{plot}}$.

Step-06:

Step-05 generates all the points for one octant.

To find the points for other seven octants, follow the eight symmetry property of circle.

This is depicted by the following figure-



EXAMPLE -MID POINT CIRCLE DRAWING ALGORITHM-

Problem-01:

Given the centre point coordinates (0, 0) and radius as 10, generate all the points to form a circle.

Solution-

Given-

- Centre Coordinates of Circle $(X_0, Y_0) = (0, 0)$
- Radius of Circle = 10

Step-01:

Assign the starting point coordinates (X_0, Y_0) as-

- $X_0 = 0$
- $Y_0 = R = 10$

Step-02:

Calculate the value of initial decision parameter P_0 as-

$$P_0 = 1 - R$$

$$P_0 = 1 - 10$$

$$P_0 = -9$$

Step-03:

As $P_{\text{initial}} < 0$, so case-01 is satisfied.

Thus,

- $X_{k+1} = X_k + 1 = 0 + 1 = 1$
- $Y_{k+1} = Y_k = 10$
- $P_{k+1} = P_k + 2 \times X_{k+1} + 1 = -9 + (2 \times 1) + 1 = -6$

Step-04:

This step is not applicable here as the given centre point coordinates is (0, 0).

Step-05:

Step-03 is executed similarly until $X_{k+1} \geq Y_{k+1}$ as follows- (we may

P_k	P_{k+1}	(X_{k+1}, Y_{k+1})
		(0, 10)
-9	-6	(1, 10)
-6	-1	(2, 10)
-1	6	(3, 10)
6	-3	(4, 9)
-3	8	(5, 9)
8	5	(6, 8)

Algorithm calculates all the points of octant-1 and terminates.

Now, the points for rest of the part are generated by following the signs of other quadrants.

The other points can also be generated by calculating each octant separately.

Here, all the points have been generated with respect to quadrant-1-

Quadrant-1 (X,Y)	Quadrant-2 (-X,Y)	Quadrant-3 (-X,-Y)	Quadrant-4 (X,-Y)
(0, 10)	(0, 10)	(0, -10)	(0, -10)
(1, 10)	(-1, 10)	(-1, -10)	(1, -10)
(2, 10)	(-2, 10)	(-2, -10)	(2, -10)
(3, 10)	(-3, 10)	(-3, -10)	(3, -10)
(4, 9)	(-4, 9)	(-4, -9)	(4, -9)
(5, 9)	(-5, 9)	(-5, -9)	(5, -9)
(6, 8)	(-6, 8)	(-6, -8)	(6, -8)
Flip octant coordinates (y,x)			
(8, 6)	(-8, 6)	(-8, -6)	(8, -6)
(9, 5)	(-9, 5)	(-9, -5)	(9, -5)
(9, 4)	(-9, 4)	(-9, -4)	(9, -4)

(10, 3)	(-10, 3)	(-10, -3)	(10, -3)
(10, 2)	(-10, 2)	(-10, -2)	(10, -2)
(10, 1)	(-10, 1)	(-10, -1)	(10, -1)
(10, 0)	(-10, 0)	(-10, 0)	(10, 0)

REFERENCE

- D. Hearn & Baker: Computer Graphics with OpenGL, Pearson Education, Third Edition, 2009.
- <https://www.geeksforgeeks.org/>
- <https://www.javatpoint.com>
- <https://www.gatevidyalay.com>
- https://www.tutorialspoint.com/computer_graphics/computer_graphics_basics.htm
- <https://studyresearch.in/2019/12/28/cohensutherlandexample/#:~:text=Clip%20a%20line%20A%20>